Reporte

Nombre: Valeria Ordaz Garcia Luis José Mendoza Serrano Omar Alejandro Salazar Castro	**Matrícula**: 2169894 2223173 2226576
Nombre del curso: Componentes de lógica del negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 5	**Actividad**: 05 - EduTechAcademy
Fecha: 04/11/2026	
Bibliografia (Formato APA): Documentos impartidos por el docente	

Objetivo:

Aplicar pruebas automáticas al núcleo de lógica de negocio de EduTechAcademy (diseñado en la Unidad 3) para demostrar, con evidencia reproducible, que sus reglas se cumplen y siguen cumpliéndose al cambiar el código: distinguir pruebas unitarias, de integración y extremo a extremo (E2E); aplicar Arrange–Act–Assert y el ciclo TDD Red–Green–Refactor; verificar persistencia y *rollback* con SQLAlchemy sin modificar datos de producción; simular cobros con *mocks*, probar autorización y revisar cobertura; e interpretar un *pipeline* de integración continua (CI).

Procedimiento:

Introducción

En la Unidad 3 diseñamos la arquitectura de EduTechAcademy (Domain Model con Service Layer, 7 Aggregates y 8 Application Services) y la acompañamos con 15 pruebas en *unittest*. Ese reporte reconoció dos límites: la suite solo demostraba lo que cubrían sus aserciones y los repositorios eran en memoria, por lo que no se había comprobado persistencia, integraciones externas ni cobertura (pendientes P-02, P-03 y P-04). En la Unidad 5 no empezamos un caso nuevo: usamos las mismas reglas del BRD para convertir esos pendientes en pruebas que se ejecutan solas, primero en la computadora del equipo y después en un *pipeline* de GitHub Actions. El código de esta entrega conserva los nombres y las reglas del diseño de la Unidad 3 (Calificacion, PorcentajeAsistencia, Cupo, Grupo, Inscripcion) y se organiza en tres capas: domain, application e infrastructure.

El trabajo avanzó en este orden: preparación del entorno, pruebas unitarias de las reglas, desarrollo guiado por pruebas (TDD) de una regla nueva, pruebas de persistencia con SQLAlchemy, simulación de la pasarela de pagos con *mocks*, pruebas de autorización por rol, análisis estático con *ruff* y cobertura con *pytest-cov*, y CI con GitHub Actions.

1. Justificación técnica

En el desarrollo de software empresarial la calidad no es una etapa final aislada: es una propiedad que se construye de forma continua. Las pruebas automáticas garantizan que las reglas críticas del dominio (certificación con 70/100 y 80 %, cupo, pago) se cumplan siempre y que un cambio nuevo no las rompa sin que nadie lo note, es decir, sin regresiones. Esto ataca de nuevo la causa de los problemas del BRD (PA-01, PA-02 y PA-04): reglas que dependen de que una persona las recuerde y las valide a mano.

Tabla 1. Herramientas seleccionadas y justificación

Herramienta	Para qué la usamos	Qué problema evita
Entorno virtual (venv)	Aísla las dependencias del proyecto del Python global.	Conflictos de versiones; “en mi equipo sí funciona”.
pytest	Descubre archivos `test_*.py`, ejecuta las aserciones `assert` y ofrece *fixtures* para preparar y limpiar el estado.	Pruebas largas y repetitivas; estado compartido entre pruebas.
unittest.mock	Sustituye la pasarela de pagos por un doble de prueba (`Mock`) con respuestas controladas.	Cobros reales o dependencia de internet durante las pruebas.
SQLAlchemy + SQLite en memoria	Prueba el mapeo y las transacciones contra una base real que vive solo durante la prueba.	Tocar datos de producción; depender de un servidor de base de datos.
ruff	Linter: revisa estilo (PEP 8) y errores comunes antes de ejecutar.	Código inconsistente; errores de sintaxis o importaciones sin uso.
pytest-cov	Mide qué líneas y ramas del código ejecutan las pruebas.	Creer que algo está probado cuando nunca se ejecutó.
GitHub Actions	Ejecuta ruff y pytest en cada *push* o *pull request*.	Que alguien suba código sin correr las pruebas.

2. Conceptos que guían las pruebas

2.1 Pirámide de pruebas: unitarias, integración y E2E

Las tres clases de prueba responden a preguntas distintas. La proporción importa: muchas pruebas pequeñas y rápidas en la base, menos de integración en el medio y pocas E2E en la punta, porque cuanto más arriba, más lentas, frágiles y costosas de mantener.

Tabla 2. Pirámide de pruebas aplicada al caso

Tipo	Qué verifica	Ejemplo en EduTechAcademy	Velocidad y cantidad
Unitaria	Una regla o clase aislada, sin base de datos ni servicios externos.	`Calificacion.aprueba()` con 69, 70 y 100; `Grupo.ocuparLugar()` sin cupo.	Milisegundos; muchas.
Integración	Que dos o más piezas funcionen juntas (dominio + repositorio + base de datos).	Guardar una `Inscripcion` con SQLAlchemy, confirmarla y recuperarla; probar `rollback`.	Rápida con SQLite en memoria; moderadas.
E2E (extremo a extremo)	Un caso de uso completo desde la entrada hasta el resultado final.	Inscribir → pagar → registrar asistencia y evaluaciones → validar certificación, invocando los Application Services.	La más lenta; pocas.

EduTechAcademy todavía no tiene interfaz gráfica ni API, así que nuestro E2E recorre el flujo completo a través del Service Layer (la capa de entrada del sistema). Una E2E con navegador queda fuera de alcance y se declara en las limitaciones. Se incluyen dos pruebas E2E (Evidencia E9).

2.2 Arrange–Act–Assert (AAA)

Cada prueba se escribe en tres bloques separados: **Arrange** (preparar los datos y el estado inicial), **Act** (ejecutar una sola acción) y **Assert** (comprobar el resultado). Una prueba con esta forma se lee como una frase, falla por una sola razón y señala exactamente qué regla se rompió.

Ejemplo de estructura AAA

```
def test_elegible_con_70_y_80_por_ciento(crear_inscripcion):
    # Arrange: calificación y asistencia justo en el límite
    insc = crear_inscripcion(calificaciones=[70], asistencias=[1, 1, 1, 1, 0])

    # Act: evaluar la certificación con el grupo finalizado
    insc.evaluarCertificacion(grupoFinalizado=True)

    # Assert: cumple ambos criterios exactamente en el umbral
    assert insc.certificacion.estado == EstadoCertificacion.ELEGIBLE
```

2.3 Ciclo TDD: Red – Green – Refactor

Tabla 3. Fases del ciclo TDD

Fase	Qué se hace	Qué se comprueba
Red (rojo)	Se escribe primero la prueba de la regla nueva, antes de que exista en el dominio.	La prueba **falla** por la razón esperada; así sabemos que la prueba sí puede detectar el error.
Green (verde)	Se agrega el código mínimo en el dominio para que la prueba pase.	La prueba nueva y todas las anteriores pasan.
Refactor	Se mejora el diseño del código (nombres, duplicación) sin cambiar su comportamiento.	La suite sigue en verde; si algo se rompe, el cambio fue incorrecto.

3. Matriz de reglas de negocio y casos de prueba

Seleccionamos 3 reglas del dominio de EduTechAcademy (certificación, cupo y pago) y para cada una diseñamos tres casos: **éxito**, **error** y **límite** (el valor exacto donde cambia la decisión). Los casos límite son los que más errores esconden: un 69/100 o un 79.9 % de asistencia no deben certificar, y un 70/100 con 80 % sí.

Tabla 4. Matriz de reglas, escenarios y casos de prueba (9 casos)

Regla del PIA	Escenario	ID del caso	Descripción del caso	Resultado esperado
RB-06 / RB-07: Certificación de alumno	Éxito	TC-CERT-01	Calificación final ≥ 70 y asistencia ≥ 80 % (por ejemplo, 90/100 y 100 %), con grupo Finalizado.	Estado: Elegible; certificado generado.
	Error	TC-CERT-02	Calificación final de 69/100 con asistencia del 100 %. (Variante: asistencia de 75 % con calificación 100.)	Estado: No elegible con MotivoNoElegibilidad; certificado denegado.
	Límite	TC-CERT-03	Calificación = 70.0 y asistencia = 80.0 %.	Estado: Elegible (límite inferior exacto).
RB-02: Cupo de grupo	Éxito	TC-CUPO-01	Inscribir a un estudiante en un grupo con cupo disponible.	Inscripción aceptada; el cupo disponible disminuye en 1.
	Error	TC-CUPO-02	Inscribir a un estudiante en un grupo con 0 lugares.	Excepción CupoAgotadoException; inscripción rechazada.
	Límite	TC-CUPO-03	Inscribir al alumno que ocupa el último lugar disponible.	Inscripción exitosa; se publica CupoAgotado y el grupo queda lleno.
RB-03 / SUP-13: Pago de inscripción	Éxito	TC-PAGO-01	Pago procesado por el monto exacto.	Inscripción Confirmada; evento PagoRecibido.
	Error	TC-PAGO-02	La pasarela de pagos rechaza la transacción.	Inscripción sigue Pendiente; no se publica PagoRecibido.

Regla del PIA	Escenario	ID del caso	Descripción del caso	Resultado esperado
	Límite	TC-PAGO-03	Curso con costo 0.00 (curso gratuito).	Se omite el cobro; inscripción Confirmada.

El modelo permite `Precio` con monto ≥ 0, pero el BRD no define qué ocurre con un curso gratuito. Registramos este comportamiento como supuesto por confirmar con el docente o el stakeholder, igual que SUP-09 a SUP-15 en la Unidad 3, y no como regla aprobada.

4. Evidencias de ejecución (paso a paso)

Evidencia E1: entorno virtual y ejecución base de pytest

Creación del entorno virtual, instalación de dependencias y primera ejecución de la suite. El entorno aísla las dependencias del sistema global; pytest detecta los archivos con prefijo `test_` y valida cada `assert`. El trabajo se realizó en Windows con PowerShell, como en las actividades anteriores.

Comandos ejecutados

```
python -m venv .venv
.venv\Scripts\Activate.ps1          # macOS/Linux: source .venv/bin/activate
pip install -r requirements.txt     # pytest, pytest-cov, ruff, sqlalchemy
python -m pytest -v
```

El archivo se escribió a mano con versiones mínimas. No conviene generarlo con "pip freeze > requirements.txt" en Windows PowerShell 5: esa redirección lo guarda en UTF-16 y "pip install -r" falla en el pipeline de GitHub Actions.

Tabla 5. Entorno de ejecución

Elemento	Valor documentado
Sistema operativo	Windows
Editor y terminal	Visual Studio Code (terminal integrada de PowerShell)
Lenguaje	Python **3.14.3**
Marco de pruebas	pytest 9.2.0 y unittest.mock (biblioteca estándar)
Persistencia	SQLAlchemy 2.1.8 con SQLite en memoria
Herramientas de calidad	ruff 0.17.3, pytest-cov 7.1.0
Comando de ejecución	`python -m pytest -v`

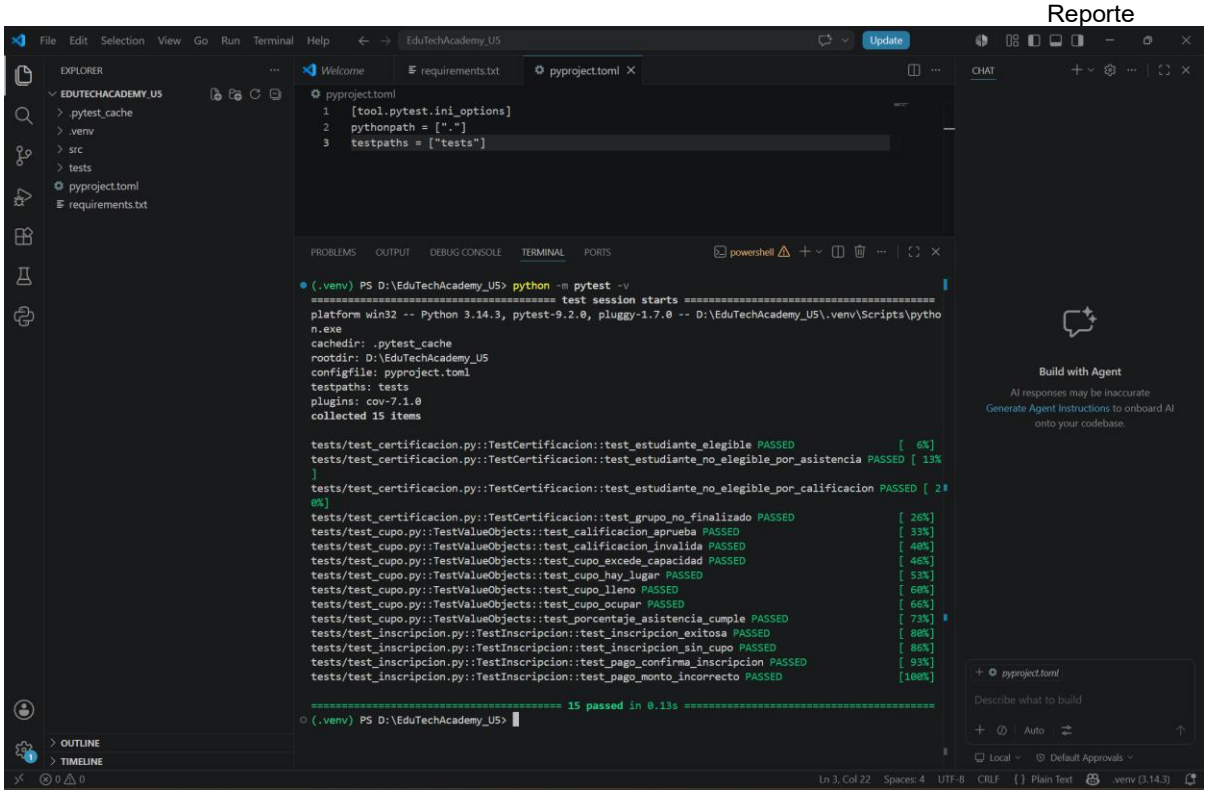


Figura 1. Creación del entorno virtual y primera ejecución de pytest.

Evidencia E2: pruebas unitarias de reglas de dominio

Pruebas unitarias sobre los Value Objects y la entidad de certificación, sin base de datos. Cada caso de la Tabla 4 (certificación y cupo) tiene su prueba; los valores del umbral se parametrizan para que 69, 70 y 100 se prueben con el mismo código.

Código de las pruebas

```
# tests/unit/test_dominio.py (fragmento; el archivo completo está en el repositorio)
import pytest

from src.domain.estados import EstadoCertificacion
from src.domain.eventos import CupoAgotado
from src.domain.excepciones import CupoAgotadoException
from src.domain.grupo import Grupo
from src.domain.valores import Calificacion, Cupo, PorcentajeAsistencia, Precio


@pytest.mark.parametrize("valor, esperado",
                         [(0, False), (69.9, False), (70, True), (100, True)])
def test_calificacion_aprueba_en_el_umbral(valor, esperado):  # RB-07
    calificacion = Calificacion(valor)     # Arrange
    resultado = calificacion.aprueba()     # Act
    assert resultado is esperado           # Assert


@pytest.mark.parametrize("valor, esperado",
                         [(0, False), (79.9, False), (80, True), (100, True)])
def test_asistencia_cumple_minimo_en_el_umbral(valor, esperado):  # RB-06
```

```
    assert PorcentajeAsistencia(valor).cumpleMinimo() is esperado


def test_cupo_no_excede_capacidad():  # TC-CUPO-02
    with pytest.raises(CupoAgotadoException):
        Cupo(capacidad=1, ocupados=1).ocupar()


def test_ultimo_lugar_publica_cupo_agotado():  # TC-CUPO-03
    grupo = Grupo("G9", capacidad=2, precio=Precio(1500), instructor_id="I1")
    grupo.ocuparLugar("E1")
    grupo.ocuparLugar("E2")                # toma el último lugar
    assert grupo.recoger_eventos() == [CupoAgotado("G9")]
    assert not grupo.cupo.hay_lugar()


def test_elegible_con_70_y_80_por_ciento(crear_inscripcion):  # TC-CERT-03
    insc = crear_inscripcion(calificaciones=[70], asistencias=[1, 1, 1, 1, 0])
    insc.evaluarCertificacion(grupoFinalizado=True)
    assert insc.certificacion.estado == EstadoCertificacion.ELEGIBLE


def test_no_elegible_con_69_y_100_por_ciento(crear_inscripcion):  # TC-CERT-02
    insc = crear_inscripcion(calificaciones=[69], asistencias=[1, 1, 1, 1, 1])
    insc.evaluarCertificacion(grupoFinalizado=True)
    assert insc.certificacion.estado == EstadoCertificacion.NO_ELEGIBLE
```

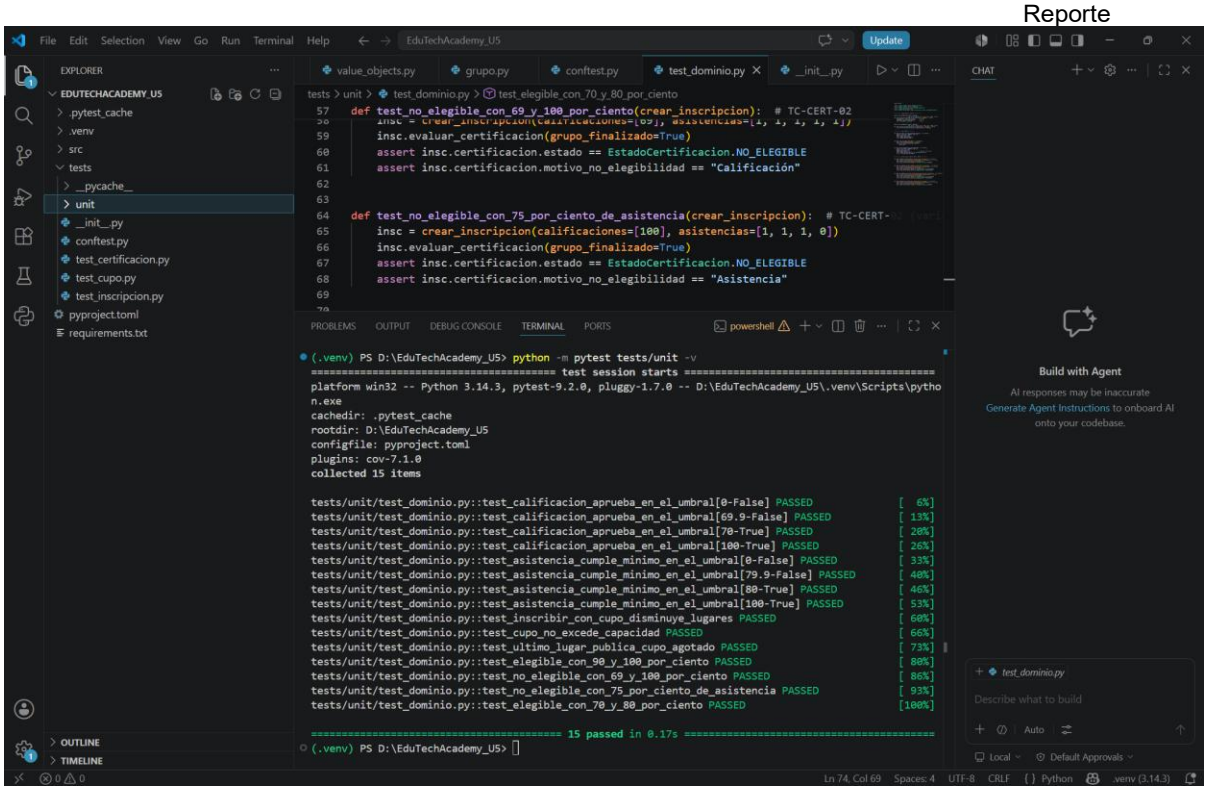


Figura 2. Ejecución exitosa de las pruebas unitarias de dominio.

Evidencia E3: desarrollo guiado por pruebas (TDD), ciclo Red / Green / Refactor

SUP-12: un estudiante no puede tener dos inscripciones activas en el mismo grupo. Esta regla era un supuesto de la Unidad 3 (pendiente de confirmar), por eso es un buen candidato: la escribimos primero como prueba y así queda definida de forma ejecutable.

Fase roja (Red): la prueba falla porque la regla no existe

```
# tests/unit/test_inscripcion_unica.py
import pytest

from src.domain.excepciones import InscripcionDuplicada


def test_rechaza_segunda_inscripcion_activa(grupo):  # SUP-12
    grupo.ocuparLugar("E1")                    # Arrange: primera inscripción
    with pytest.raises(InscripcionDuplicada):  # Assert: debe rechazarse
        grupo.ocuparLugar("E1")                # Act: segunda inscripción
```

Resultado esperado en rojo: como `Grupo.ocuparLugar()` todavía acepta la segunda inscripción, pytest reporta `Failed: DID NOT RAISE` y la prueba queda en **FAILED**. La excepción InscripcionDuplicada ya existe en el módulo de excepciones; por eso la falla es DID NOT RAISE y no un ImportError: la prueba debe fallar por la regla que falta, no por un nombre inexistente.

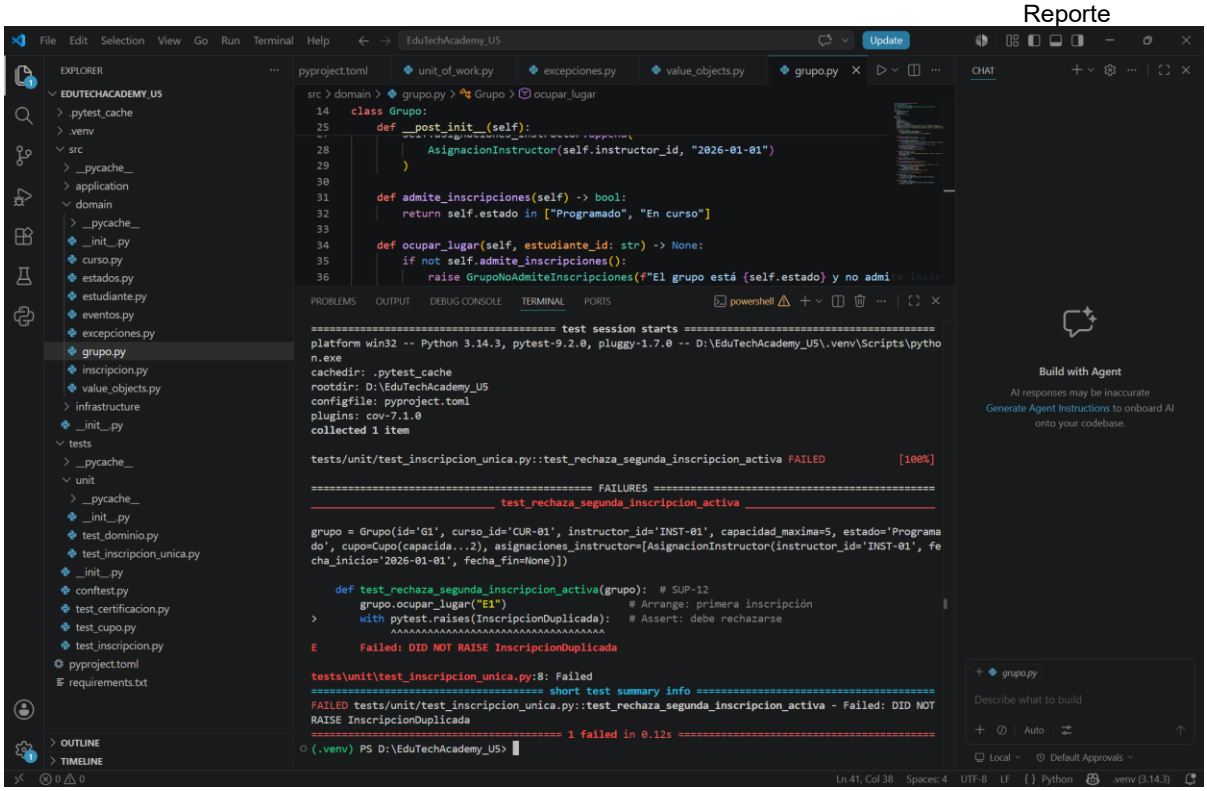


Figura 3. Fase roja: la prueba de SUP-12 falla porque la regla aún no existe.

Implementación y fase verde (Green)

```
# src/domain/grupo.py (cambio mínimo; se muestra solo lo relevante)
from src.domain.excepciones import GrupoNoAdmiteInscripciones, InscripcionDuplicada

def ocuparLugar(self, estudiante_id):
    ...                                          # (validación del estado del grupo)
    if estudiante_id in self._estudiantes_activos:
        raise InscripcionDuplicada(estudiante_id, self.id)   # SUP-12
    self.cupo = self.cupo.ocupar()                            # RB-02
    self._estudiantes_activos.add(estudiante_id)
    ...                                          # (publicar CupoAgotado si se llenó)
```

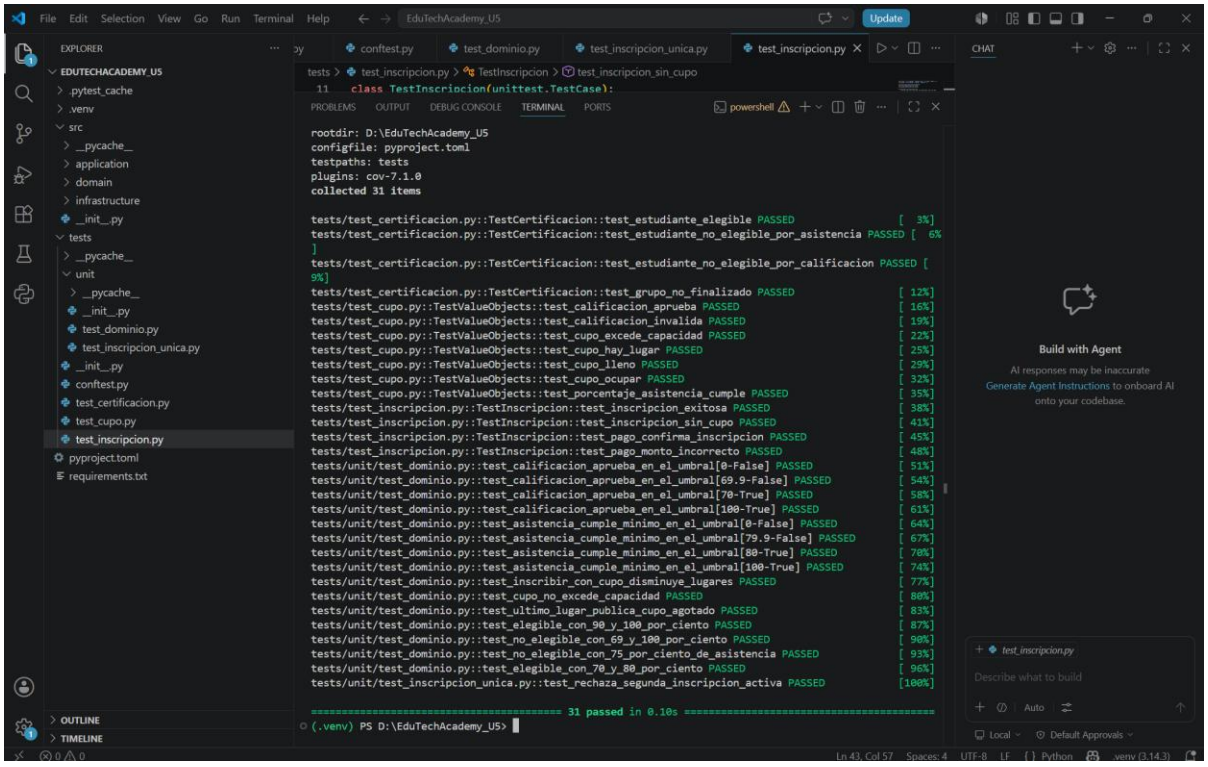


Figura 4. Fase verde: la prueba pasa y no se rompió ninguna prueba anterior.

Refactor

Con la suite en verde se puede mejorar el código sin miedo: por ejemplo, mover la verificación de duplicados a un método privado con nombre del lenguaje ubicuo (`_ya_esta_inscrito`). Después del cambio se vuelve a ejecutar `pytest`; si todo sigue en verde, el comportamiento no cambió.

Evidencia E4: persistencia con SQLAlchemy y SQLite en memoria

Validación del mapeo relacional y del ciclo de vida de la transacción. Es una prueba de **integración**: participan el dominio, el repositorio y una base de datos real. Para no modificar datos de producción, la base es `sqlite:///:memory:`: se crea al empezar cada prueba y desaparece al terminar. Las pruebas nunca leen la cadena de conexión de producción. El mapeo guarda la inscripción en la tabla inscripciones (id, estudiante, grupo, estado y precio en centavos enteros, para no usar float). SUP-12 se refleja en la base con un índice único parcial que solo aplica a inscripciones no canceladas.

Tabla 6. Ciclo de vida de una transacción en SQLAlchemy

Operación	Qué hace
commit()	Confirma de forma permanente los cambios de la transacción actual y los hace visibles para otras sesiones.
rollback()	Aborta la transacción actual y revierte lo no confirmado, devolviendo la base a su último estado consistente ante una excepción o un error.

Operación	Qué hace
flush()	Envía los cambios a la base dentro de la transacción **sin** confirmarlos; sirve para detectar errores de restricciones antes del `commit`.

Fixture de sesión aislada

```
# tests/conftest.py (fixture de sesión)
import pytest
from sqlalchemy import create_engine
from sqlalchemy.orm import Session

from src.infrastructure.orm import Base


@pytest.fixture()
def session():
    engine = create_engine("sqlite:///:memory:")   # nunca la BD real
    Base.metadata.create_all(engine)
    with Session(engine) as s:
        yield s
        s.rollback()                               # limpia lo no confirmado
    engine.dispose()                               # la BD en memoria desaparece
```

Código de las pruebas de integración

```
# tests/integration/test_persistencia.py (fragmento)
import pytest
from sqlalchemy.exc import IntegrityError

from src.domain.estados import EstadoInscripcion
from src.domain.inscripcion import Inscripcion
from src.domain.valores import Precio
from src.infrastructure.repositorio_sql import InscripcionRepositorySQL


def test_guardar_y_recuperar_inscripcion(session):
    repo = InscripcionRepositorySQL(session)                  # Arrange
    insc = Inscripcion.nueva("E1", "G1", precio=Precio(1500))

    repo.guardar(insc)                                        # Act
    session.commit()
    recuperada = repo.buscar(insc.id)

    assert recuperada.estado == EstadoInscripcion.PENDIENTE   # Assert
    assert recuperada.precio == Precio(1500)


def test_rollback_descarta_cambios_no_confirmados(session):
    repo = InscripcionRepositorySQL(session)
    insc = Inscripcion.nueva("E2", "G1", precio=Precio(1500))

    repo.guardar(insc)
    session.flush()          # visible solo dentro de la transacción
    session.rollback()       # se revierte
```

```
    assert repo.buscar(insc.id) is None


def test_error_de_restriccion_deja_la_base_consistente(session):
    repo = InscripcionRepositorySQL(session)
    repo.guardar(Inscripcion.nueva("E3", "G1", Precio(1500)))
    session.commit()

    with pytest.raises(IntegrityError):    # SUP-12 también en la BD
        repo.guardar(Inscripcion.nueva("E3", "G1", Precio(1500)))
        session.commit()
    session.rollback()

    assert len(repo.por_estudiante("E3")) == 1
```

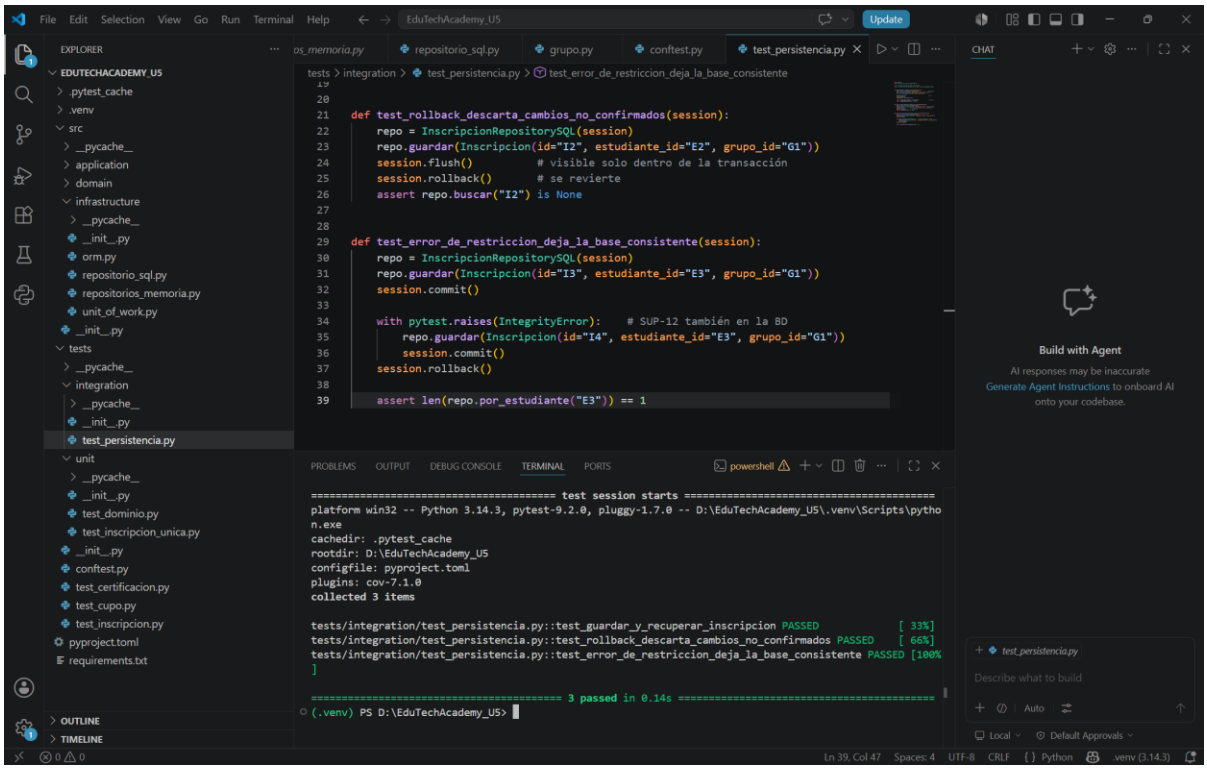


Figura 5. Pruebas de integración con SQLAlchemy y SQLite en memoria.

Evidencia E5: simulación de la pasarela de pagos con mocks

El servicio de pago se prueba con `unittest.mock`: la pasarela real se sustituye por un doble que devuelve lo que decidamos. Así probamos los tres resultados posibles sin hacer cobros reales y sin depender de internet.

Confirmar la inscripción en la base de datos local **antes** de recibir la respuesta de la pasarela puede dejar alumnos dados de alta sin pago confirmado o ingresos perdidos. Por eso el orden del servicio es: cobrar → si la pasarela aprueba, confirmar la inscripción y guardar en una sola transacción → publicar `PagoRecibido`. Si la pasarela rechaza o no responde, la inscripción sigue Pendiente (SUP-13) y no se publica ningún evento.

Tabla 7. Escenarios de pago simulados

Escenario	Doble de prueba	Resultado esperado	Qué demuestra
Pago aprobado	`pasarela.cobrar.return_value = ResultadoCobro(aprobado=True)`	Inscripción Confirmada; evento PagoRecibido.	RB-03 cumplida.
Pago rechazado	`pasarela.cobrar.return_value = ResultadoCobro(aprobado=False)`	Inscripción Pendiente; sin PagoRecibido.	SUP-13: el rechazo no cambia el estado.
Timeout de conexión	`pasarela.cobrar.side_effect = TimeoutError`	Error controlado; inscripción Pendiente; sin PagoRecibido.	No se confirma sin respuesta de la pasarela.

Código de las pruebas con mocks (`insc` es una fixture con una inscripción Pendiente; pasarela y publicador son Mock)

```
# tests/integration/test_pago_mock.py (fragmento)
import pytest

from src.application.errores import PasarelaNoDisponible
from src.application.puertos import ResultadoCobro
from src.application.servicios import PagoService
from src.domain.estados import EstadoInscripcion
from src.domain.eventos import PagoRecibido


# TC-PAGO-01
def test_pago_aprobado_confirma(insc, repo, pasarela, publicador):
    pasarela.cobrar.return_value = ResultadoCobro(aprobado=True)   # Arrange
    servicio = PagoService(pasarela, repo, publicador)

    servicio.registrarPago(insc.id, monto=insc.precio)             # Act

    pasarela.cobrar.assert_called_once_with(insc.id, insc.precio)  # Assert
    assert repo.buscar(insc.id).estado == EstadoInscripcion.CONFIRMADA
    publicador.publicar.assert_called_once()
    assert isinstance(publicador.publicar.call_args.args[0], PagoRecibido)


# TC-PAGO-02
def test_pago_rechazado_deja_pendiente(insc, repo, pasarela, publicador):
    pasarela.cobrar.return_value = ResultadoCobro(aprobado=False)
    servicio = PagoService(pasarela, repo, publicador)

    servicio.registrar_pago(insc.id, monto=insc.precio)

    assert repo.buscar(insc.id).estado == EstadoInscripcion.PENDIENTE
    publicador.publicar.assert_not_called()


def test_timeout_no_confirma(insc, repo, pasarela, publicador):
    pasarela.cobrar.side_effect = TimeoutError
    servicio = PagoService(pasarela, repo, publicador)

    with pytest.raises(PasarelaNoDisponible):
```

```
    servicio.registrar_pago(insc.id, monto=insc.precio)

assert repo.buscar(insc.id).estado == EstadoInscripcion.PENDIENTE
publicador.publicar.assert_not_called()
```

El curso gratuito (TC-PAGO-03) comprueba que la pasarela no se llama; un monto distinto al precio se rechaza antes de cobrar; y un segundo pago de una inscripción ya Confirmada se rechaza antes de llegar a la pasarela, que se llama una sola vez (evita cobrar dos veces). En el proyecto no existe ninguna implementación real de PasarelaPagos: la única que usan las pruebas es el Mock, por lo que no es posible generar un cargo real.

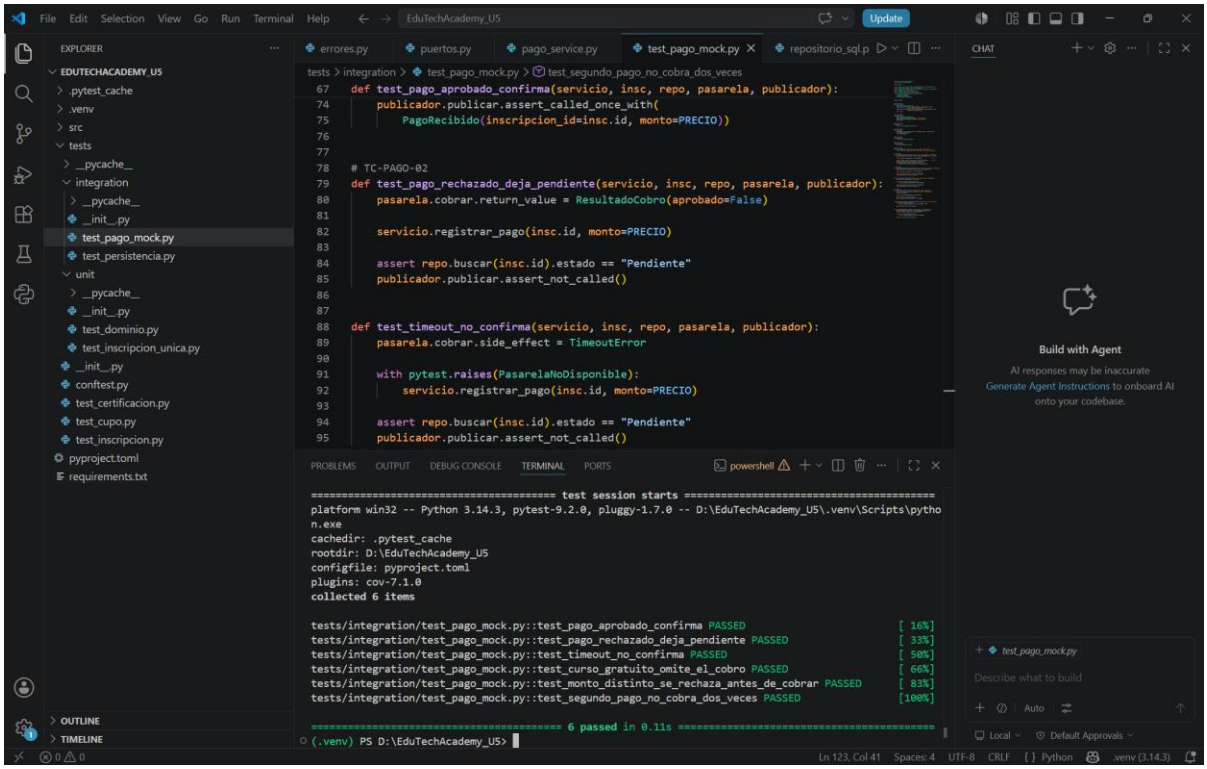


Figura 6. Pruebas del servicio de pago con la pasarela simulada.

Evidencia E6: matriz y pruebas de autorización y control de acceso (RBAC)

RB-14: cada actor tiene permisos distintos. Según el ADR-005, los permisos se verifican en el Service Layer y no dentro del dominio; por eso las pruebas se hacen sobre los servicios.

Tabla 8. Matriz de permisos por rol (RBAC)

Rol	Inscribirse a un curso	Registrar asistencia	Validar y emitir certificación
Estudiante	✓ (solo para sí mismo)	✗	✗
Instructor	✗	✓ (solo en sus grupos asignados)	✗
Control Escolar	✓ (para cualquier estudiante)	✗	✓

Código de las pruebas de autorización

```
# tests/integration/test_autorizacion.py (fragmento)
import pytest

from src.application.errores import PermisoDenegado


@pytest.mark.parametrize("actor, operacion, permitido", [
    ("estudiante",       "inscribirse_a_si_mismo", True),
    ("estudiante",       "inscribir_a_otro",       False),
    ("estudiante",       "registrar_asistencia",   False),
    ("estudiante",       "emitir_certificado",     False),
    ("instructor",       "registrar_asistencia",   True),   # grupo propio
    ("instructor_ajeno", "registrar_asistencia",   False),  # grupo ajeno
    ("instructor",       "inscribir_a_otro",       False),
    ("instructor",       "emitir_certificado",     False),
    ("control_escolar",  "inscribir_a_otro",       True),
    ("control_escolar",  "emitir_certificado",     True),
    ("control_escolar",  "registrar_asistencia",   False),
])
def test_permisos_por_rol(servicios, actor, operacion, permitido):  # RB-14
    antes = _escrituras(servicios)

    if permitido:
        ejecutar(servicios, actor, operacion)       # no lanza error
    else:
        with pytest.raises(PermisoDenegado):
            ejecutar(servicios, actor, operacion)
        assert _escrituras(servicios) == antes      # nada se modificó
```

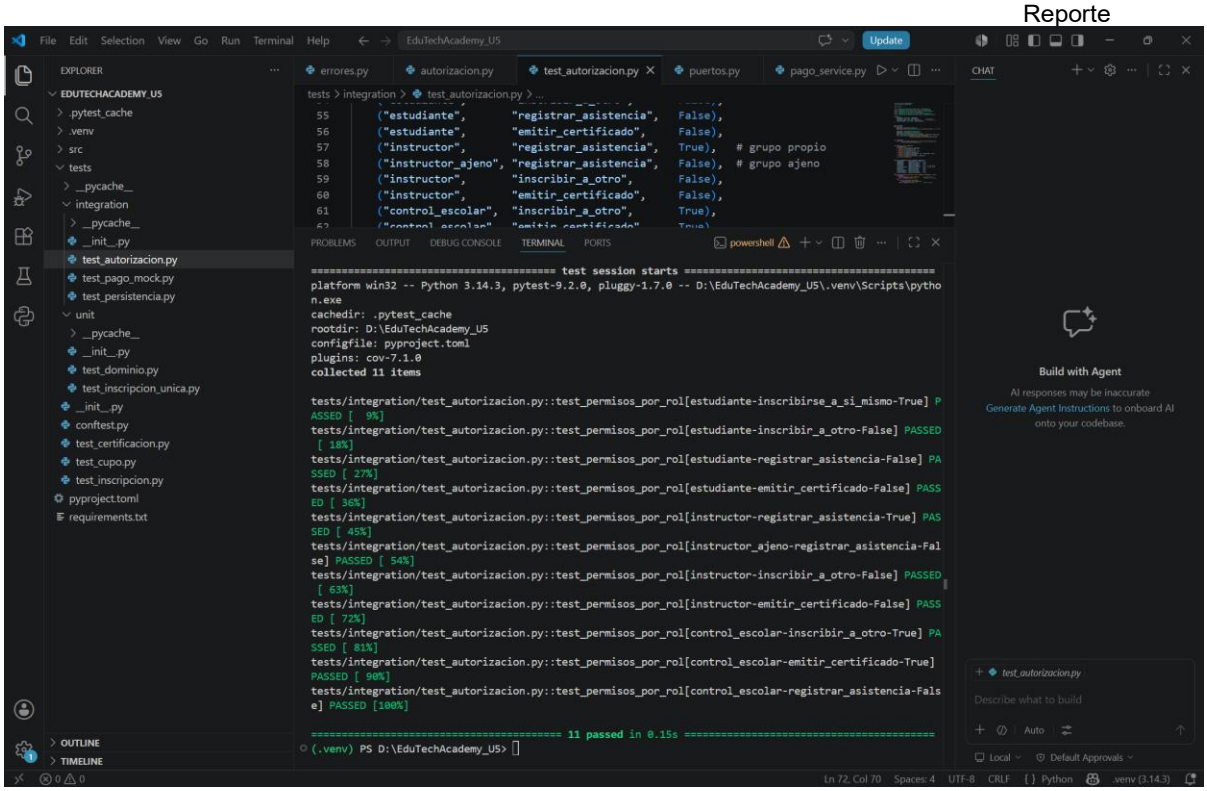


Figura 7. Pruebas de autorización por rol (RB-14).

Evidencia E7: análisis estático (ruff) y cobertura (pytest-cov)

`ruff` revisa el código sin ejecutarlo: estilo, importaciones sin uso y errores comunes. `pytest-cov` mide qué porcentaje del código ejecutan las pruebas; con `--cov-branch` también cuenta si se recorrieron ambos lados de cada `if`. Fijamos un mínimo de 80 % para que el *pipeline* falle si baja.

Comandos ejecutados

```
ruff check . python -m pytest --cov=src --cov-branch --cov-report=term-missing --cov-report=html --cov-fail-under=80
```

Ruff reportó 31 hallazgos en la primera ejecución (importaciones desordenadas o sin usar y anotaciones List/Optional anticuadas), que se corrigieron y la segunda ejecución salió sin errores; cobertura total 90.20 % (mínimo exigido: 80 %), con 87 pruebas en verde. Módulo con menor cobertura: unit_of_work.py (0 %), porque ninguna prueba lo usa todavía y no participa en las reglas probadas en esta unidad; le sigue grupo.py (69 %), cuyos métodos de liberar lugar y de manejo de estados no tienen prueba propia.

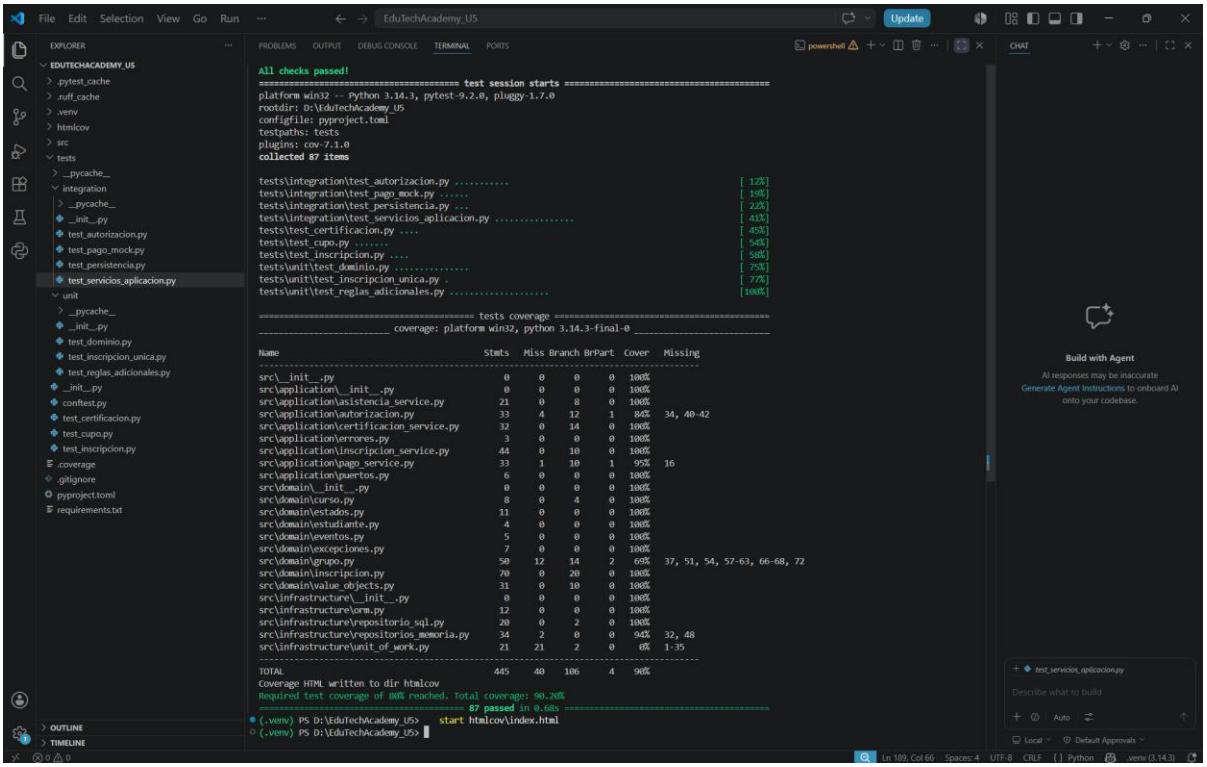


Figura 8. Resultado de ruff y de la cobertura en terminal.

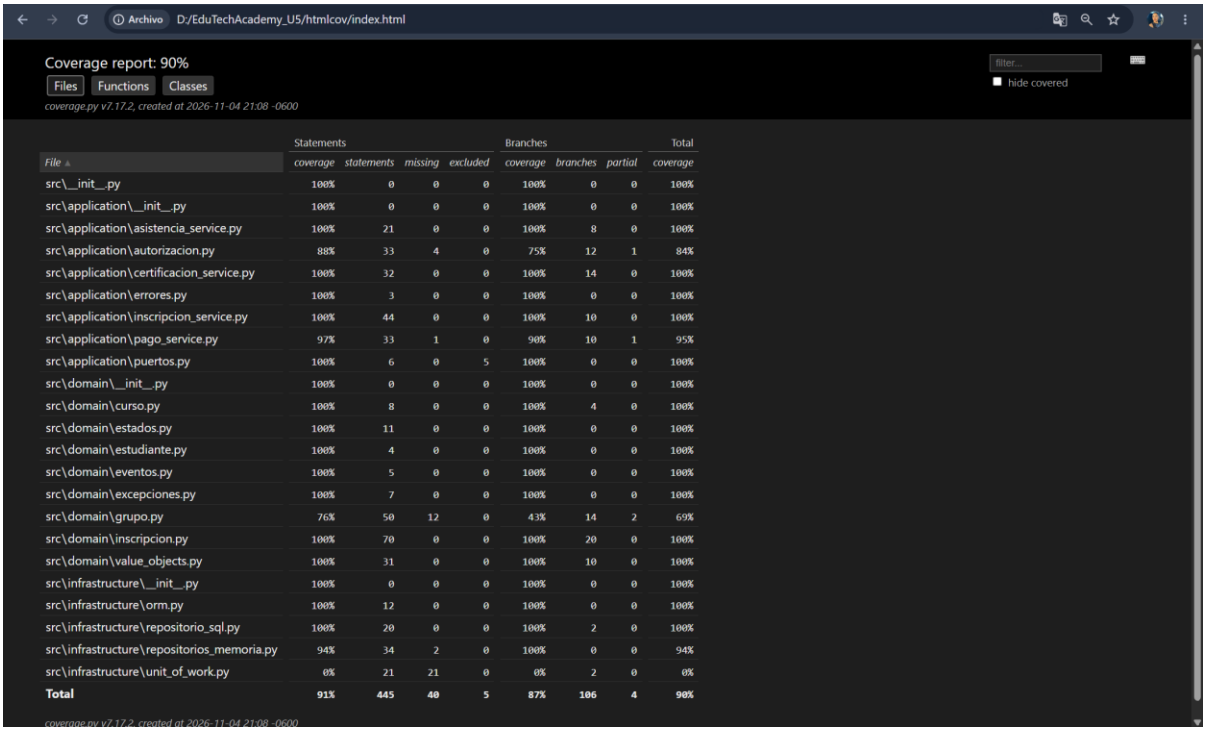


Figura 9. Reporte HTML de cobertura (htmlcov/index.html).

> Un porcentaje alto indica que el código se ejecutó durante las pruebas, no que las aserciones sean suficientes. Una prueba sin `assert` sube la cobertura sin comprobar nada. Por eso la cobertura se usa para encontrar código **sin probar**, no para afirmar que el sistema es correcto.

Evidencia E8: integración continua con GitHub Actions

Archivo del pipeline: `.github/workflows/ci.yml`. En cada *push* y *pull request* a `main`, GitHub crea una máquina limpia, instala las dependencias desde `requirements.txt` y ejecuta ruff y pytest con cobertura. Como el resultado no depende de la computadora de quien programa, cualquier persona puede reproducirlo. El pipeline usa Python 3.12; el código es compatible con 3.10 o superior.

.github/workflows/ci.yml

```yaml
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  calidad:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -r requirements.txt
      # 1) análisis estático
      - run: ruff check .
      # 2) pruebas y cobertura mínima
      - run: python -m pytest --cov=src --cov-branch --cov-fail-under=80
```

Los pasos se ejecutan en orden y el primero que falla detiene el trabajo y lo marca en rojo. Un fallo en `ruff check` indica un problema de estilo o error estático; un fallo en `pytest` indica que una regla del negocio se rompió o que una prueba falló; y un fallo por `--cov-fail-under` indica que la cobertura bajó del mínimo.

Tabla 9. Secuencia de verificación del pipeline

#	Ejecución	Qué se hizo	Resultado esperado
1	Pipeline inicial	Subir el código con las pruebas y el `ci.yml`.	Verde

#	Ejecución	Qué se hizo	Resultado esperado
2	Fallo provocado a propósito	“Cambiar el umbral de Calificacion.aprueba() de >= 70 a >= 75 en src/domain/value_objects.py y subirlo; fallan 3 pruebas: test_calificacion_aprueba (Unidad 3), test_calificacion_aprueba_en_el_umbral[70-True] y TC-CERT-03 (test_elegible_con_70_y_80_por_ciento).”	Rojo: falla pytest, se identifica la regla rota
3	Arreglo final	Revertir el umbral a 70 y volver a subir.	Verde

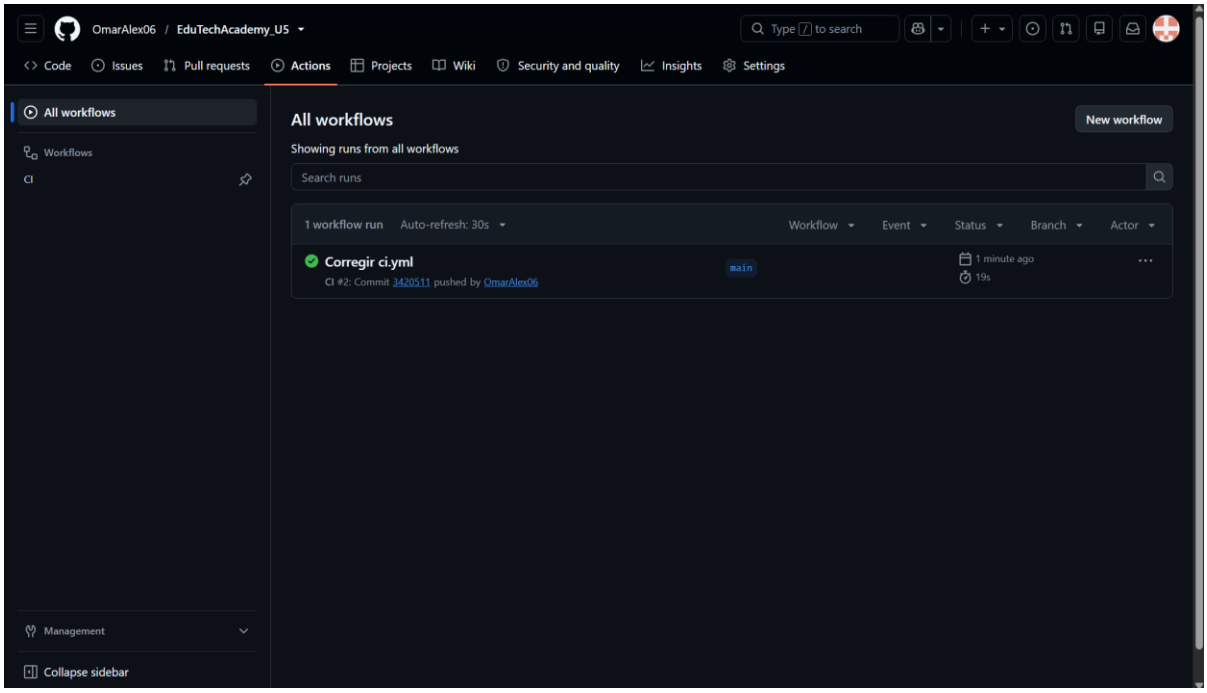


Figura 10. Ejecución del pipeline en verde.

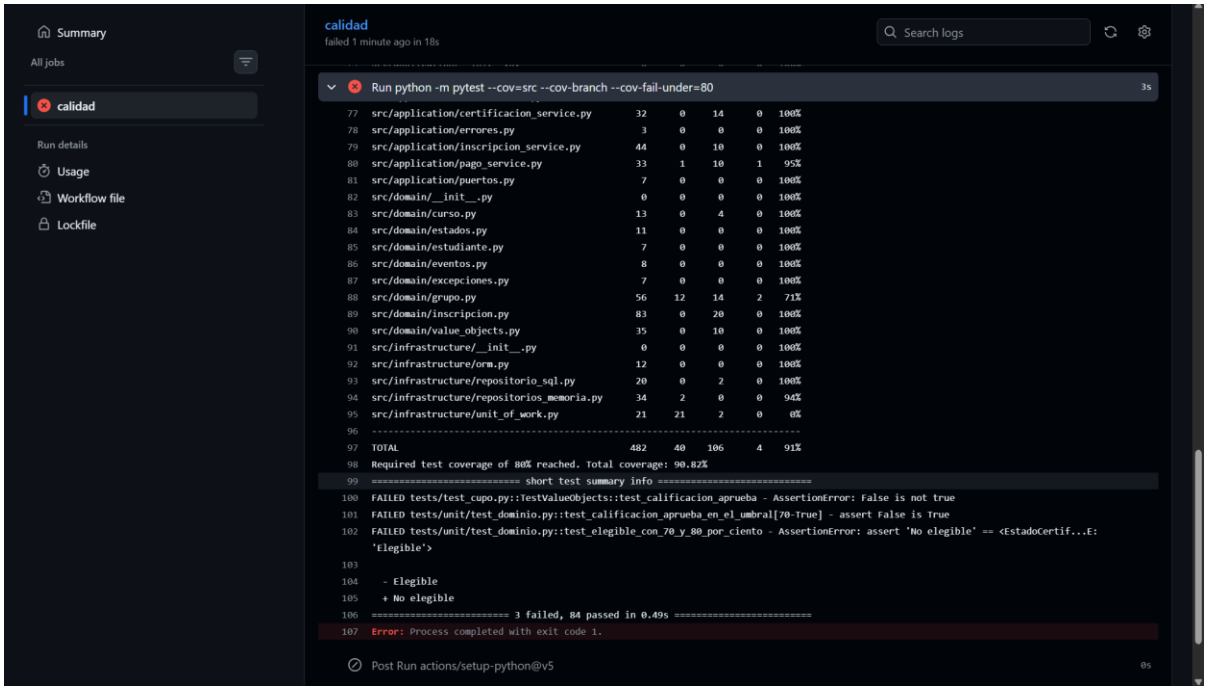


Figura 11. Fallo provocado a propósito (rojo).

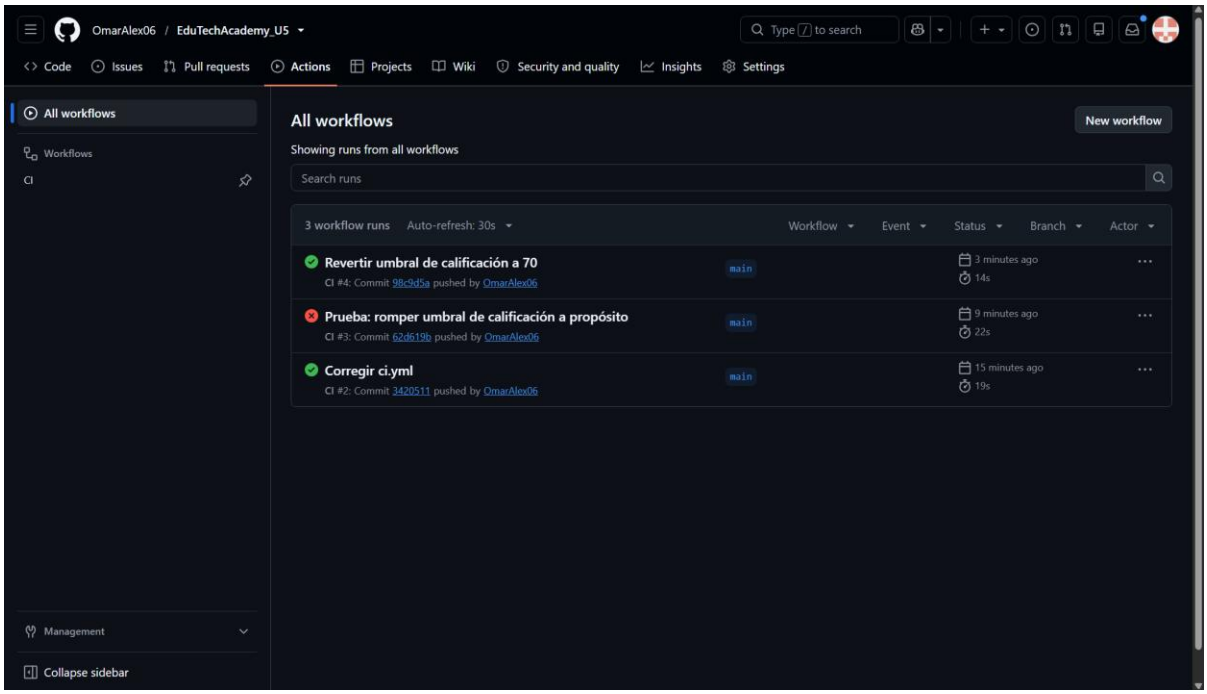


Figura 12. Pipeline en verde tras el arreglo final.

Evidencia E9: prueba E2E por el Service Layer y matriz CSV

Dos pruebas E2E recorren el caso de uso completo a través de los Application Services, con repositorios en memoria y la pasarela simulada: (1) un alumno se inscribe, paga, asiste 4 de 5 sesiones con calificación 70 y queda Elegible justo en los umbrales; (2) un pago rechazado deja la inscripción Pendiente y por eso el instructor no puede registrar asistencia (RB-04). El archivo matriz_casos.csv del repositorio reúne los 9 casos de la Tabla 4 más 3 casos adicionales (SUP-12, doble cobro y permisos), cada uno con su prueba pytest y una columna estado que se llena con el resultado de la ejecución.

Código de la prueba E2E principal

```
# tests/e2e/test_flujo_completo.py (fragmento)
def test_alumno_cumple_todo_y_queda_elegible(servicios):
    servicios.pasarela.cobrar.return_value = ResultadoCobro(aprobado=True)

    # 1) Se inscribe -> queda Pendiente
    insc = servicios.inscripcion.inscribir("E1", "G1")

    # 2) Paga -> la pasarela aprueba, se Confirma y se publica PagoRecibido
    servicios.pago.registra_pago(insc.id, insc.precio)

    # 3) El instructor registra 5 sesiones (4 asistencias = 80 %) y una nota de 70
    for presente in (True, True, True, True, False):
        servicios.asistencia.registrarAsistencia(insc.id, presente)
    servicios.asistencia.registrarEvaluacion(insc.id, 70)

    # 4) Control Escolar finaliza el grupo y valida la certificación
    servicios.grupo_service.grupo.finalizar("G1")
    servicios.certificacion.validarCertificacion("G1")

    # 5) Justo en los umbrales (70/100 y 80 %) queda Elegible
    assert insc.certificacion.estado == EstadoCertificacion.ELEGIBLE
```

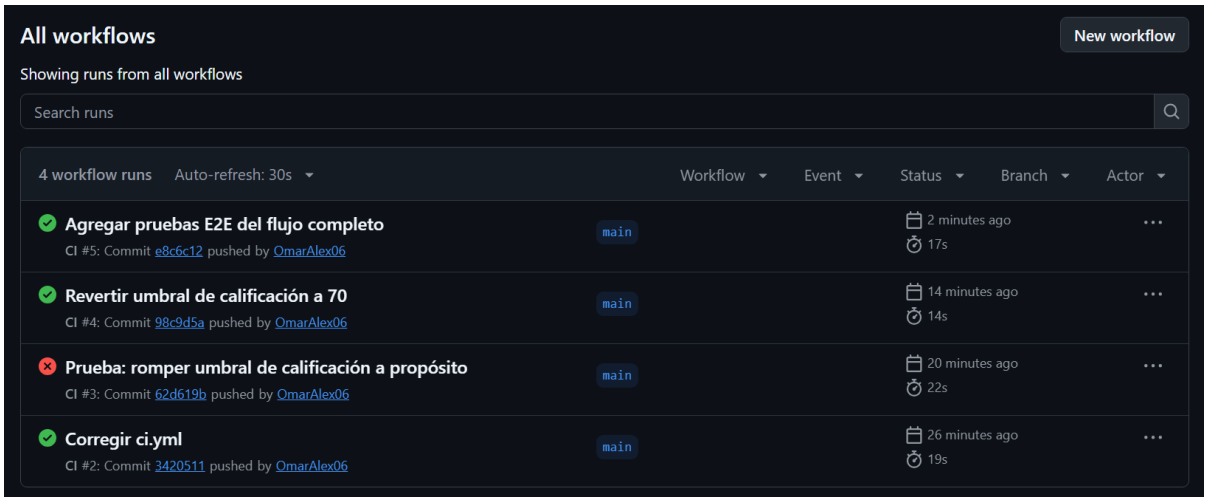


Figura 13. Pruebas E2E del flujo completo a través del Service Layer.

5. Preguntas del reporte

¿En qué se distinguen las pruebas unitarias, de integración y E2E? La unitaria prueba una regla aislada (por ejemplo `Calificacion.aprueba()`); la de integración prueba que varias piezas funcionen juntas (dominio, repositorio y base de datos); la E2E recorre un caso de uso completo. Se prefieren muchas unitarias porque son rápidas y señalan con precisión dónde está el error; las E2E se reservan para pocos flujos críticos.

¿Por qué usar Arrange–Act–Assert? Separa preparar, ejecutar y comprobar. Cada prueba verifica una sola acción, es fácil de leer y, cuando falla, el bloque que falló indica si el problema estaba en los datos, en la operación o en el resultado.

¿Por qué en TDD hay que ver fallar la prueba primero? Porque una prueba que nunca ha fallado no demuestra que pueda detectar el error. Ver el rojo confirma que la prueba comprueba lo que dice y que el código nuevo es la causa de que pase.

¿Cómo se prueba la persistencia sin tocar datos de producción? Con una base SQLite en memoria creada y destruida en cada prueba mediante una *fixture*, sin leer nunca la conexión de producción. La limitación es que SQLite no es idéntico al motor de producción (tipos, bloqueos, comportamiento de restricciones), por lo que se declara que falta probar contra el motor real.

¿Para qué sirve un mock y qué no demuestra? Permite controlar la respuesta de un servicio externo (aprobado, rechazado, timeout) sin cobrar de verdad. No demuestra que la integración real funcione: si la pasarela cambia su contrato, el mock seguirá pasando. Eso requiere pruebas contra un ambiente de pruebas (*sandbox*) del proveedor.

¿Una cobertura del 100 % garantiza que el sistema es correcto? No. Mide qué código se ejecutó, no si las aserciones son suficientes ni si se probaron los casos límite. Sirve para descubrir código que nunca se prueba.

¿Qué aporta el pipeline de CI? Ejecuta las mismas verificaciones en una máquina limpia en cada cambio. Evita que alguien suba código sin correr las pruebas y deja evidencia pública de quién rompió qué y cuándo se arregló.

6. Limitaciones y pendientes

Una suite en verde demuestra solo los comportamientos que cubren sus aserciones. Para ser claros sobre el alcance de esta entrega, la Tabla 10 reúne lo que las pruebas **no** demuestran.

Tabla 10. Limitaciones y pendientes abiertos

ID	Pendiente o limitación	Estado
P-01	Confirmar los supuestos SUP-09 a SUP-15 y el comportamiento del curso gratuito (TC-PAGO-03) con el docente o el stakeholder.	Pendiente de confirmación.
P-02	SQLite en memoria no equivale al motor de producción; falta ejecutar la integración contra el motor real.	Pendiente de implementación.
P-03	Los *mocks* no prueban la pasarela real; falta una prueba contra el ambiente *sandbox* del proveedor.	Pendiente de implementación.
P-04	No hay prueba de concurrencia del control optimista de versión en Grupo (ADR-006).	Pendiente de ampliación.

ID	Pendiente o limitación	Estado
P-05	No hay E2E con interfaz gráfica ni navegador; el E2E recorre el Service Layer.	Fuera de alcance.
P-06	El repositorio SQL solo mapea la inscripción (id, estudiante, grupo, estado y precio); las asistencias y evaluaciones viven solo en memoria.	Pendiente de ampliación.

7. Problemas encontrados y aprendizajes

1. **Problema 1: la regla nueva de TDD rompió una prueba de la Unidad 3.** Al llegar a la fase verde de SUP-12 (un estudiante no puede tener dos inscripciones activas en el mismo grupo), pytest terminó con 1 fallida y 30 aprobadas. La prueba test_inscripcion_sin_cupo de la Unidad 3 inscribía dos veces a EST-01 para “simular otro estudiante”, y ahora el dominio lanzaba InscripcionDuplicada. La causa era un error de datos en la prueba vieja, que la regla nueva dejó al descubierto. Se corrigió usando un segundo estudiante distinto en esa prueba y la suite volvió a verde. Aprendimos que una prueba de regresión puede detectar errores en las pruebas anteriores, no solo en el código, y que conviene correr toda la suite en cada paso del ciclo TDD.

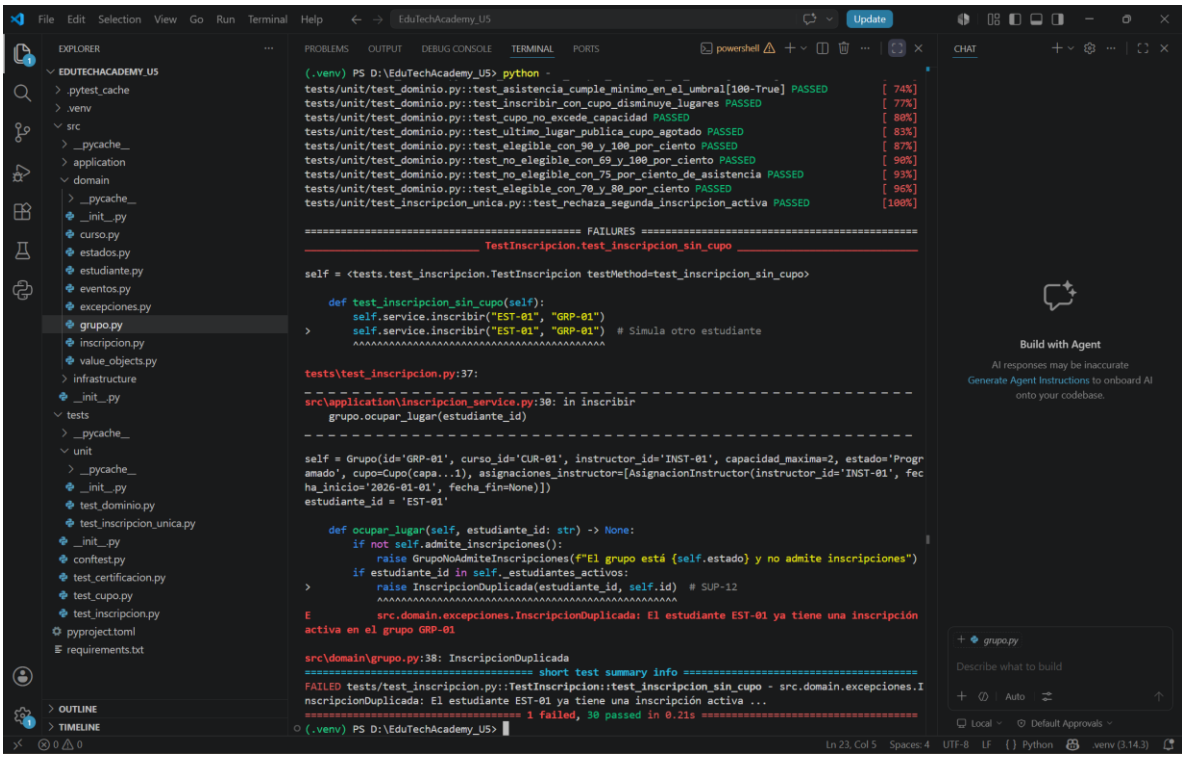


Figura 14. Falla de test_inscripcion_sin_cupo al agregar SUP-12 (1 failed, 30 passed).

2. **Problema 2: 31 hallazgos en el primer ruff check ..** El linter marcó importaciones desordenadas, importaciones sin usar y anotaciones anticuadas (List, Optional). 28 eran corregibles con ruff check . --fix y los demás se arreglaron a mano. Después volvimos a correr pytest para confirmar que ninguna regla cambió, y ruff salió con All checks passed!. Aprendimos que el análisis estático conviene correrlo desde el inicio y no al final

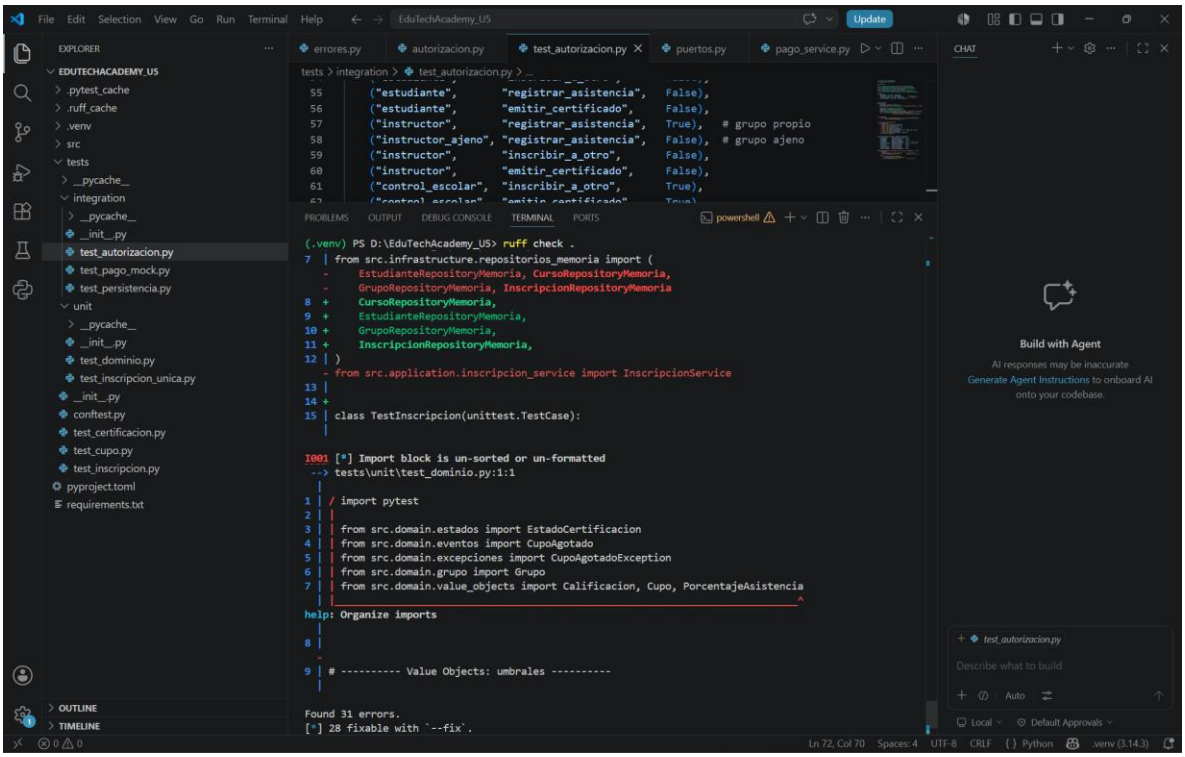


Figura 15. Primera ejecución de ruff check . con 31 hallazgos.

8. Uso responsable de IA (bitácora)

Se usó IA para planear la actividad, revisar el borrador del reporte y generar el código base de las pruebas. Ningún resultado se aceptó sin ejecutarse: las pruebas se corrieron en nuestras computadoras y en GitHub Actions. Agregamos una fila por cada caso en que la IA propuso algo que corregimos o rechazamos.

Tabla 11. Bitácora de uso de IA

#	Objetivo y prompt	Respuesta útil y error o supuesto detectado	Decisión del equipo
1	Entender el laboratorio y planear la actividad a partir de la Unidad 3. Prompt: revisar los archivos de la unidad y explicar qué se debe hacer.	Resumen de los 9 pasos del laboratorio y propuesta de 3 reglas del PIA (certificación, cupo y pago). Supuesto: la IA no pudo abrir el repositorio de la Unidad 3 (GitHub bloqueó el acceso automático), así que propuso el diseño sin ver el código original.	Se aceptó el plan de tres reglas. Como la IA no vio el código original, el equipo creó el proyecto de la Unidad 5 con los mismos nombres y reglas de la Unidad 3: Calificacion, PorcentajeAsistencia, Cupo, Grupo e Inscripcion, organizado en domain, application e infrastructure.
2	Revisar el borrador de este reporte y generar el código de las pruebas. Prompt: “checa si está bien, corrige los errores y ayúdame con el código paso a paso”.	Detectó fragmentos sin importaciones ni fixtures, un assert que comparaba una clase con una instancia (PagoRecibido), el riesgo de “pip freeze >” en PowerShell (UTF-16), una fixture con llaves foráneas inexistentes y la falta de evidencia E2E. Supuesto: la IA no pudo ejecutar pytest ni SQLAlchemy; solo verificó la lógica con un corredor propio.	Se aceptaron las correcciones de importaciones y fixtures y el aviso sobre pip freeze, por lo que el requirements.txt se escribió a mano. Al ejecutar, la regla SUP-12 rompió test_inscripcion_sin_cupo y se arregló con un segundo estudiante; ruff marcó 31 hallazgos y se corrigieron hasta ver All checks passed. Al final pytest y GitHub Actions quedaron en verde.
3	Adaptar a nuestro proyecto el código de ejemplo que propuso la IA para las pruebas de persistencia y de pagos.	Propuso pruebas escritas para un dominio genérico. Error detectado: algunos nombres y tipos no coincidían con nuestro código real, por ejemplo los estados, que en nuestro proyecto son texto y no un enum. Supuesto: la IA no podía ver ni ejecutar nuestro repositorio.	Se rechazó copiar el ejemplo tal cual. Se adaptaron los nombres y se revisó cada prueba contra el código real. Evidencia: pytest en verde en local y en GitHub Actions.

9. Conclusión

El defecto más costoso en producción sería certificar a un estudiante que no cumple los criterios o dejar inscrito a alguien cuyo pago nunca se confirmó. Un certificado emitido por error daña la reputación de la academia y de quienes sí lo merecen, y retirarlo después genera conflictos con alumnos y empresas clientes; un cobro no confirmado significa ingresos perdidos que nadie nota hasta una conciliación. Ambos defectos pasarían inadvertidos porque el sistema seguiría funcionando sin

mostrar errores. Para el primero, la mejor defensa son los casos límite TC-CERT-03 (70/100 y 80 %) y TC-CERT-02 (69/100 con 100 % de asistencia): si alguien cambia un umbral, la suite y el pipeline de GitHub Actions fallan antes de que el cambio llegue a main. Para el segundo, las pruebas con la pasarela simulada (pago rechazado y timeout) comprueban que la inscripción sigue Pendiente y que no se publica PagoRecibido. Que un programa se ejecute sin errores no demuestra que sus reglas se cumplan.

10. Conclusiones

Conclusión general

Con esta actividad comprendimos que una arquitectura bien diseñada no basta: necesita pruebas que demuestren, cada vez que el código cambia, que las reglas del negocio siguen cumpliéndose. Las pruebas unitarias protegen las reglas con sus valores límite (70/100, 80 %, último lugar del cupo), las de integración comprueban que la persistencia y el *rollback* dejan la base consistente sin tocar datos reales, y los *mocks* permiten probar cobros aprobados, rechazados y con *timeout* sin depender de un proveedor externo.

También aprendimos que escribir la prueba antes (TDD) obliga a definir con precisión qué significa una regla, y que la cobertura y el análisis estático sirven para encontrar huecos, no para presumir calidad. Finalmente, el *pipeline* de CI convierte todo lo anterior en una verificación que no depende de la memoria ni de la computadora de una persona: al provocar un fallo a propósito y arreglarlo vimos el ciclo completo. Este diseño sigue siendo un prototipo; lo que queda por probar contra un motor de base de datos y una pasarela reales está documentado en la sección de limitaciones.

Valeria Ordaz García, matrícula 2169894

Lo que más aprendí fue que una prueba que pasa no dice nada hasta que se ve fallar. Cuando cambiamos a propósito el umbral de calificación de 70 a 75, el pipeline se puso en rojo y señaló tres pruebas rotas, incluida una de la Unidad 3 que ya existía. Ahí entendí para qué sirve una suite: protege las reglas aunque alguien se olvide de ellas. Lo más difícil fue ver que la primera cobertura salió en 72.96 % y que subirla exigía pruebas con aserciones reales, no solo ejecutar más código. Lo resolvimos probando los errores y los límites de cada servicio, hasta llegar a 90.20 %.

Luis José Mendoza Serrano, matrícula 2223173

Lo más difícil para mí fue GitHub Actions. La primera ejecución falló antes de correr nada, con el mensaje “No event triggers defined in on”, porque el archivo ci.yml tenía mal la indentación. Aprendí que YAML es muy sensible a los espacios y que conviene leer el error exacto en la pestaña Actions en lugar de adivinar. Después de corregirlo, el pipeline pasó en verde y entendí su valor: el resultado ya no depende de mi computadora. También me quedó claro que los mocks permiten probar pagos aprobados, rechazados y con timeout sin cobrar de verdad, aunque no demuestran que la pasarela real funcione.

Omar Alejandro Salazar Castro, matrícula 2226576

Aprendí a trabajar con el código que ya existía en lugar de empezar de cero. Las pruebas de

persistencia con SQLAlchemy y SQLite en memoria me enseñaron la diferencia entre flush, commit y rollback: en la prueba del rollback, un cambio visible dentro de la transacción desaparece al revertirla. Lo más difícil fue que parte del código de ejemplo del reporte no coincidía con nuestro dominio (por ejemplo, los estados son texto y no un enum), así que tuvimos que adaptar los nombres y revisar cada prueba contra el código real. Además, ruff marcó 31 hallazgos de importaciones y anotaciones que corregimos antes de medir la cobertura.

Link de repositorio de GitHub:

https://github.com/OmarAlex06/EduTechAcademy_U5